

functions for pipeline

Qun Li

22 Aug 2025 at 10:12:37

Contents

1	pipelines for mapping scRNA-seq and scATAC data	2
2	pipelines for porcessing scRNA-seq data	3
3	pipelines for processing scATAC-seq data	5
4	pipelines for trajectory analysis of scATAC-seq data	9
5	pipelines for generating single-cell somatic mutation data list	13
6	pipelines for generating sample stage mutation similarity matrix	14
7	pipelines for generating mutgeneExp	16
8	pipelines for generating mutpeakgenedata	18
9	pipelines for stage diff muts	21
10	pipelines for generating celltype similarity (scRNA vs scATAC)	23
11	pipelines for generating celltype similarity (scRNA)	25
12	pipelines for generating celltype similarity (scATAC)	26
13	pipelines for generating celltype similarity by stages(scRNA)	27
14	pipelines for generating celltype similarity by stages(scATA)	29
15	pipelines for generating mutation similarity between two stages	30
16	pipelines for generating mutation similarity between two stages for lineage switching analysis	34

1 pipelines for mapping scRNA-seq and scATAC data

```
#####
# Example SLURM script
# Run cellranger count for scRNA-seq data in server
#####

#!/bin/bash
#SBATCH -A naiss2024-22-89
#SBATCH --job-name cellCountATAC
#SBATCH --partition shared
#SBATCH -t 0-06:00:00
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --output /cfs/klemming/home/q/qunli/qunl/log/log.%j.%x
#SBATCH --error /cfs/klemming/home/q/qunli/qunl/log/error.%j.%x
#SBATCH --mail-type ALL
#SBATCH --mail-user qun.li@ki.se

ml bioinfo-tools
ml cellranger/7.1.0

refFA=/cfs/klemming/home/q/qunli/qundir/reference/homo/GRCh38_maskMito/GRCh38/
workingDir=/cfs/klemming/home/q/qunli/qunl/Qun/callVariants/mito/fastq/

for i in CD34_500_Day08 CD34_500_Day14 CD34_800_Day08 CD34_800_Day14 CD34_800_Day20
do
    sampleID=${i}
    cellranger count --id=${sampleID} --fastqs=${workingDir}/${sampleID}/ --transcriptome=${refFA} --s
done

#####
# Example SLURM script
# Run cellranger-atac count for scATAC-seq data in server
#####

#!/bin/bash
#SBATCH -A naiss2024-22-89
#SBATCH --job-name cellCountATAC
#SBATCH --partition shared
#SBATCH -t 0-06:00:00
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --output /cfs/klemming/home/q/qunli/qunl/log/log.%j.%x
#SBATCH --error /cfs/klemming/home/q/qunli/qunl/log/error.%j.%x
#SBATCH --mail-type ALL
```

```
#SBATCH --mail-user qun.li@ki.se
```

```
ml bioinfo-tools
```

```
ml cellranger-ATAC/2.1.0
```

```
refFA=/cfs/klemming/home/q/qunli/qundir/reference/homo/GRCh38_maskMito/GRCh38/
```

```
workingDir=/cfs/klemming/home/q/qunli/qunl/Qun/callVariants/mito/fastq/
```

```
for i in CD34_500_Day08 CD34_500_Day14 CD34_800_Day08 CD34_800_Day14 CD34_800_Day20
do
```

```
    sampleID=${i}
```

```
    cellranger-atac count --id=${sampleID} --fastqs=${workingDir}/${sampleID}/ --reference=${refFA} --
```

```
done
```

2 pipelines for porcessing scRNA-seq data

```
# load libraries
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
library(Seurat)
```

```
library(harmony)
```

```
library(Signac)
```

```
library(ArchR)
```

```
# get scRNA matrix paths
```

```
get_matrix_scRNA_path <- function(sampleID, matrixDir){
```

```
    tag <- paste0(sampleID, "_processed")
```

```
    fileNames <- list.files(matrixDir)
```

```
    file_barcode <- paste0(matrixDir, grep(paste0(tag, "_barcodes"), fileNames, value = TRUE))
```

```
    file_matrix <- paste0(matrixDir, grep(paste0(tag, "_matrix"), fileNames, value = TRUE))
```

```
    file_genes <- paste0(matrixDir, grep(paste0(tag, "_genes"), fileNames, value = TRUE))
```

```
    file_metadata <- paste0(matrixDir, grep(paste0(tag, "_metadata"), fileNames, value = TRUE))
```

```
    res <- list(barcode = file_barcode, matrix = file_matrix, genes = file_genes, metadata = file_metadata)
```

```
    return(res)
```

```
}
```

```
# load scRNA data and process seurat object
```

```
loadData_scRNA <- function(sampleID, matrixDir, tagForData, dims = 1:20){
```

```
    print(paste0("process: ", sampleID))
```

```
    # mtx
```

```
    print(" load mtx ..")
```

```
    input_items <- get_matrix_scRNA_path(sampleID, matrixDir)
```

```
    barcode_tmp_path <- input_items$barcode
```

```
    matrix_tmp_path <- input_items$matrix
```

```
    genes_tmp_path <- input_items$genes
```

```
    metadata_tmp_path <- input_items$metadata
```

```
    if (tagForData == "processed"){
```

```
        data <- ReadMtx(mtx = matrix_tmp_path, features = genes_tmp_path, cells = barcode_tmp_path, feature
```

```
    }
```

```
    if (tagForData == "raw"){
```

```
        data <- ReadMtx(mtx = matrix_tmp_path, features = genes_tmp_path, cells = barcode_tmp_path)
```

```

}
seurat_obj <- CreateSeuratObject(counts = data, min.cells = 3, min.features = 200, project = sampleID)
print("  load meta ..")
metadata_df <- read.delim(gzfile(metadata_tmp_path), sep = "\t", header = TRUE, row.names = 1, stringsAsFactors = FALSE)
metadata_df <- metadata_df[colnames(seurat_obj), , drop = FALSE]
seurat_obj <- AddMetaData(seurat_obj, metadata_df)
seurat_obj[["percent.mt"]] <- PercentageFeatureSet(seurat_obj, pattern = "^MT-")
seurat_obj <- subset(seurat_obj, subset = nCount_RNA > 1000 & nFeature_RNA > 500 & percent.mt < 10)
print("  process seurat obj")
seurat_obj <- seurat_obj %>%
  NormalizeData() %>%
  FindVariableFeatures() %>%
  ScaleData() %>%
  RunPCA() %>%
  RunUMAP(dims = dims)

return(seurat_obj)
}

# run harmony integration on multiple samples
run_harmony_samples <- function(sample_list, dims = 1:20){
  for (sample in names(sample_list)) {
    sample_list[[sample]]$sampleID <- sample
  }
  combined_seurat <- merge(sample_list[[1]], y = sample_list[-1])
  combined_seurat <- NormalizeData(combined_seurat)
  combined_seurat <- FindVariableFeatures(combined_seurat)
  combined_seurat <- ScaleData(combined_seurat)
  combined_seurat <- RunPCA(combined_seurat)
  combined_seurat <- RunHarmony(combined_seurat, group.by.vars = "orig.ident")
  combined_seurat <- FindNeighbors(combined_seurat, dims = dims, reduction = 'harmony')
  combined_seurat <- FindClusters(combined_seurat, resolution = 0.5)
  combined_seurat <- RunUMAP(combined_seurat, reduction = "harmony", dims = dims, return.model = TRUE)
  return(combined_seurat)
}

# save harmony integrated seurat object
save_harmony_rds <- function(sample_list, pathOut){
  HSPC <- c("HSC", "Progenitor")
  Myeloid <- c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
  Lymphoid <- c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive", "CD4.Memory", "CD8.Naive", "CD8.Memory", "CI")
  Other <- c("Early.Erythrocyte", "Late.Erythrocyte", "Early.Basophil", "Plasma", "Unknown")

  seurat_harmony <- run_harmony_samples(sample_list)

  seurat_harmony@meta.data$QCellType <- ifelse(seurat_harmony@meta.data$Classified_Celltype %in% HSPC,
                                              ifelse(seurat_harmony@meta.data$Classified_Celltype %in% Myeloid, "Myeloid",
                                              ifelse(seurat_harmony@meta.data$Classified_Celltype %in% Lymphoid, "Lymphoid",
                                              "Other")), "Other")

  saveRDS(object = seurat_harmony, file = pathOut)
}

# run pipeline for all patients

```

```

# set paths and sample info
matrixDir = "/Users/liqun/scMutraceProject/Data/EGA_AML/data/matrix/scRNA/"

scRNA_list <- list(
  AML1 = c("AML1_DX", "AML1_REM"),
  AML2 = c("AML2_DX", "AML2_REM", "AML2_REL"),
  AML3 = c("AML3_DX", "AML3_REM"),
  AML4 = c("AML4_DX", "AML4_REL"),
  AML5 = c("AML5_DX", "AML5_REM", "AML5_REL"),
  AML6 = c("AML6_DX", "AML6_REM", "AML6_REL"),
  AML7 = c("AML7_REM", "AML7_REL"),
  AML8 = c("AML8_DX", "AML8_REM", "AML8_REL"),
  AML9 = c("AML9_DX", "AML9_REM", "AML9_REL"),
  AML10 = c("AML10_DX", "AML10_REM", "AML10_REL"),
  AML11 = c("AML11_DX", "AML11_REM", "AML11_REL"),
  AML12 = c("AML12_DX", "AML12_REM", "AML12_REL"),
  AML13 = c("AML13_DX", "AML13_REM", "AML13_REL"),
  AML14 = c("AML14_DX", "AML14_REM"),
  AML15 = c("AML15_DX", "AML15_REM", "AML15_REL"),
  AML16 = c("AML16_DX", "AML16_REM", "AML16_REL"),
  AML17 = c("AML17_DX", "AML17_REL"),
  AML18 = c("AML18_DX", "AML18_REL"),
  AML19 = c("AML19_DX", "AML19_REL"),
  AML20 = c("AML20_DX", "AML20_REM"),
  AML21 = c("AML21_DX", "AML21_REM", "AML21_REL"),
  AML22 = c("AML22_DX", "AML22_REM", "AML22_REL"),
  AML23 = c("AML23_DX", "AML23_REM", "AML23_REL"),
  AML24 = c("AML24_DX", "AML24_REM", "AML24_REL"),
  AML25 = c("AML25_DX", "AML25_REM", "AML25_REL"),
  AML26 = c("AML26_DX", "AML26_REM", "AML26_REL"),
  AML27 = c("AML27_DX", "AML27_REM", "AML27_REL"),
  AML28 = c("AML28_REM", "AML28_REL")
)

# process each patient
for (patient in names(scRNA_list)) {
  sample_names <- scRNA_list[[patient]]
  sample_list <- lapply(sample_names, function(sample) {
    loadData_scRNA(sample, matrixDir, "processed")
  })
  names(sample_list) <- sample_names
  out_path <- paste0("../data/rds/", patient, "_scRNA.rds")
  save_harmony_rds(sample_list, pathOut = out_path)
}

```

3 pipelines for processing scATAC-seq data

```

library(ArchR)
library(Seurat)
library(magrittr)
library(dplyr)

```

```

library(patchwork)
library(stringr)
library(hdf5r)
library(Matrix)
library(org.Hs.eg.db)
library(BSgenome.Hsapiens.UCSC.hg38)
library(TxDb.Hsapiens.UCSC.hg38.knownGene)

addArchRGenome("hg38")
addArchRThreads(threads = 10)

options(ArchR.downloadPath = "/cfs/klemming/home/q/qunli/qundir/AML/data/ArchR")
scATAC_path_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scATA/"
scATAC_output_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scATAOut/"
scrNA_RDS_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scrNARDS/"
MACS2_location <- "/pdc/software/other/macs2/2.2.9.1/bin/macs2"
dbArchR <- "/cfs/klemming/home/q/qunli/qundir/AML/data/ArchR/Annotations/ArchR-Hg38-v1.Anno"
scATAC_file_names <- list.files(scATAC_path_dir)

sample_selected_scATA <- list(
  AML1 = c("AML1_DX", "AML1_REM"),
  AML2 = c("AML2_DX", "AML2_REM", "AML2_REL"),
  AML3 = c("AML3_DX", "AML3_REM"),
  AML4 = c("AML4_DX", "AML4_REL"),
  AML5 = c("AML5_DX", "AML5_REM", "AML5_REL"),
  AML6 = c("AML6_DX", "AML6_REL"),
  AML8 = c("AML8_DX", "AML8_REM", "AML8_REL"),
  AML10 = c("AML10_DX", "AML10_REM", "AML10_REL"),
  AML11 = c("AML11_DX", "AML11_REM", "AML11_REL"),
  AML12 = c("AML12_DX", "AML12_REL"),
  AML13 = c("AML13_DX", "AML13_REL"),
  AML15 = c("AML15_DX", "AML15_REM", "AML15_REL"),
  AML16 = c("AML16_DX", "AML16_REM", "AML16_REL"),
  AML17 = c("AML17_DX", "AML17_REL"),
  AML18 = c("AML18_DX", "AML18_REL"),
  AML19 = c("AML19_DX", "AML19_REL"),
  AML20 = c("AML20_DX", "AML20_REM"),
  AML21 = c("AML21_DX", "AML21_REM", "AML21_REL"),
  AML22 = c("AML22_DX", "AML22_REL"),
  AML24 = c("AML24_DX", "AML24_REM", "AML24_REL"),
  AML25 = c("AML25_DX", "AML25_REM", "AML25_REL"),
  AML26 = c("AML26_DX", "AML26_REM", "AML26_REL"),
  AML27 = c("AML27_DX", "AML27_REM", "AML27_REL")
)

# sort and gzip for each sample
# read and sort
for (sample in names(sample_selected_scATA)){
  for (stage in sample_selected_scATA[[sample]]){
    message(paste(sample, stage, sep = "_"))
    raw_fragment_file_path <- paste0(scATAC_path_dir, grep(paste0(stage, "_filtered_fragments"), sc
    frag <- fread(raw_fragment_file_path, header = TRUE)[, .(seqnames, start, end, RG)]
    setorder(frag, seqnames, start)
  }
}

```

```

    # write output (no header)
    frag_sort <- paste0(scATAC_path_dir, stage, "_filtered_fragments.sorted.tsv")
    fwrite(frag, frag_sort, sep = "\t", col.names = FALSE)
    # bgzip + tabix
    system(paste("bgzip -f", frag_sort))
    system(paste("tabix -p bed", paste0(frag_sort, ".gz")))
  }
}

setwd(scATAC_output_dir)

# create createArrowFiles
for (sample in names(sample_selected_scATA)){
  message(paste0("create arrowfiles.. ", sample))
  inputFiles_sorted <- c()
  for (stage in sample_selected_scATA[[sample]]){
    frag_sort <- paste0(scATAC_path_dir, stage, "_Qun_fragment.sorted.tsv.gz")
    inputFiles_sorted[stage] <- frag_sort
  }
  tmp_out_dir <- paste0(scATAC_output_dir, sample, "/")
  dir.create(tmp_out_dir, recursive = TRUE)
  ArrowFiles <- createArrowFiles(inputFiles = inputFiles_sorted, sampleNames = names(inputFiles_sorted))
  scATAC_data <- ArchRProject(ArrowFiles = ArrowFiles, outputDirectory = tmp_out_dir)
  # get genescore matrix
  geneScoreMat <- getMatrixFromProject(ArchRProj = scATAC_data, useMatrix = "GeneScoreMatrix")
  mat <- assay(geneScoreMat)
  rownames(mat) <- rowData(geneScoreMat)$name
  colnames(mat) <- rownames(colData(geneScoreMat))
  df_geneScore <- as.data.frame(as.matrix(t(mat)))

  saveRDS(df_geneScore, paste0(tmp_out_dir, sample, "_geneScore.rds"))

  doubletscores <- addDoubletScores(input = scATAC_data, k = 10, knnMethod = "UMAP", LSIMethod = 1)
  scATAC_data <- filterDoublets(ArchRProj = doubletscores)
  scATAC_data <- addIterativeLSI(ArchRProj = scATAC_data, useMatrix = "TileMatrix", name = "IterativeLSI")
  scATAC_data <- addClusters(input = scATAC_data, reducedDims = "IterativeLSI", force=T)
  scATAC_data <- addUMAP(ArchRProj = scATAC_data, reducedDims = "IterativeLSI", force=T)
  scATAC_data <- addTSNE(ArchRProj = scATAC_data, reducedDims = "IterativeLSI", name = "TSNELSI", perplexity = 30)

  scATAC_data <- addImputeWeights(scATAC_data)
  scATAC_data <- addGroupCoverages(ArchRProj = scATAC_data, groupBy = "Clusters", force= T)
  scATAC_data <- addReproduciblePeakSet(ArchRProj = scATAC_data, groupBy = "Clusters", pathToMacs2 = MACS2_PATH,
                                       minCells = 50, method = "p", cutOff = 1e-5, peakMethod = "Macs2")
  scATAC_data <- addPeakMatrix(scATAC_data)
  scATAC_data <- addMotifAnnotations(ArchRProj = scATAC_data, motifSet = "cisbp", name = "Motif", force=TRUE)
  scATAC_data <- addBgdPeaks(scATAC_data)
  scATAC_data <- addDeviationsMatrix(ArchRProj = scATAC_data, peakAnnotation = "Motif", force = TRUE)
  scATAC_data <- addArchRAnnotations(ArchRProj = scATAC_data, collection = "ATAC", db = dbArchR, force = TRUE)

  #saveArchRProject(ArchRProj = scATAC_data, outputDirectory = tmp_out_dir, load = FALSE)

  # create v4 seurat for RNA
  scRNA_data <- readRDS(paste0(scRNA_RDS_dir, sample, "_scRNA.rds"))

```

```

meta_scrNA <- scrNA_data@meta.data
DefaultAssay(scrNA_data) <- "RNA"
layer_names <- grep("^counts", Layers(scrNA_data, assay = "RNA"), value = TRUE)
rna_counts_list <- lapply(layer_names, function(l) {
  LayerData(scrNA_data, assay = "RNA", layer = l)
})

all_genes <- unique(unlist(lapply(rna_counts_list, rownames)))

rna_counts_list_aligned <- lapply(rna_counts_list, function(m) {
  m2 <- Matrix::Matrix(0, nrow = length(all_genes), ncol = ncol(m), sparse = TRUE)
  rownames(m2) <- all_genes
  colnames(m2) <- colnames(m)
  m2[rownames(m), ] <- m
  return(m2)
})

rna_counts <- do.call(cbind, rna_counts_list_aligned)
scrNA_data_v4 <- CreateSeuratObject(counts = rna_counts)
scrNA_data_v4@meta.data <- scrNA_data@meta.data

# integrate
#scATAC_data <- loadArchRProject(tmp_out_dir)
scATAC_data <- addGeneIntegrationMatrix(ArchRProj = scATAC_data, useMatrix = "GeneScoreMatrix", matrix = rna_counts,
  reducedDims = "IterativeLSI", seRNA = scrNA_data_v4, addToArr = TRUE,
  nameCell = "predictedCell_Un", nameGroup = "predictedGroup_Un")

scATAC_data <- addPeak2GeneLinks(
  ArchRProj = scATAC_data,
  reducedDims = "IterativeLSI"
)

p2g <- getPeak2GeneLinks(
  ArchRProj = scATAC_data,
  corCutOff = 0.3,
  resolution = 1,
  returnLoops = FALSE
)

p2g$geneName <- mcols(metadata(p2g)$geneSet)$name[p2g$idxRNA]
p2g$peakName <- (metadata(p2g)$peakSet %>% {paste0(seqnames(.), "_", start(.), "_", end(.))})[p2g$idxRNA]

markersGS <- getMarkerFeatures(
  ArchRProj = scATAC_data,
  useMatrix = "GeneScoreMatrix",
  groupBy = "predictedGroup_Un",
  bias = c("TSSEnrichment", "log10(nFragments)"),
  testMethod = "wilcoxon"
)

markerList <- getMarkers(markersGS, cutOff = "FDR <= 0.01 & Log2FC >= 1.25")

# geneexp
gene_exp_mat <- getMatrixFromProject(scATAC_data, useMatrix = "GeneIntegrationMatrix")
gene_exp_mat_mat <- as.matrix(assay(gene_exp_mat))

```



```

gene_names <- rowData(gene_exp_mat)$name
rownames(gene_exp_mat_mat) <- gene_names

# peakcell
peak_matrix <- getMatrixFromProject(ArchRProj = scATAC_data, useMatrix = "PeakMatrix")
peakxcell <- assay(peak_matrix)
peak_names <- paste0(seqnames(peak_matrix), ":", start(peak_matrix), "-", end(peak_matrix))
peak_cellnames <- rownames(colData(peak_matrix))
df_peakCell <- as.data.frame(as.matrix(peakxcell))
rownames(df_peakCell) <- peak_names
colnames(df_peakCell) <- peak_cellnames

# motif
# peak_motif
pSet <- getPeakSet(ArchRProj = scATAC_data)
pSet$name <- paste(seqnames(pSet), paste(start(pSet), end(pSet), sep = "-"), sep = ":")
matches <- getMatches(ArchRProj = scATAC_data, name = "Motif")
rownames(matches) <- paste(seqnames(matches), paste(start(matches), end(matches), sep = "-"), sep = ":")
matches <- matches[pSet$name]
matches_peakMotif_mat <- as.data.frame(as.matrix(assay(matches)))

# cell_motif
motif_mat <- getMatrixFromProject(scATAC_data, useMatrix = "MotifMatrix")
motif_matrix <- assay(motif_mat)
motif_names <- rowData(motif_mat)$name
rownames(motif_matrix) <- motif_names
motif_cellMotif_df <- as.data.frame(motif_matrix)

# save outputs
saveRDS(markersGS, paste0(tmp_out_dir, sample, "_markersGS.rds"))
saveRDS(markerList, paste0(tmp_out_dir, sample, "_markerList.rds"))
saveRDS(as.data.frame(p2g), paste0(tmp_out_dir, sample, "_peak2Genes.rds"))
saveRDS(as.data.frame(gene_exp_mat_mat), paste0(tmp_out_dir, sample, "_geneExp.rds"))
saveRDS(df_peakCell, paste0(tmp_out_dir, sample, "_peakCell.rds"))
saveRDS(matches_peakMotif_mat, paste0(tmp_out_dir, sample, "_peakMotif.rds"))
saveRDS(motif_cellMotif_df, paste0(tmp_out_dir, sample, "_cellMotif.rds"))
saveArchRProject(ArchRProj = scATAC_data, outputDirectory = tmp_out_dir, load = FALSE)
}

```

4 pipelines for trajectory analysis of scATAC-seq data

```

library(ArchR)
library(Seurat)
library(magrittr)
library(dplyr)
library(patchwork)
library(stringr)
library(hdf5r)
library(Matrix)
library(org.Hs.eg.db)
library(BSgenome.Hsapiens.UCSC.hg38)

```

```

library(TxDb.Hsapiens.UCSC.hg38.knownGene)

addArchRGenome("hg38")
addArchRThreads(threads = 10)

options(ArchR.downloadPath = "/cfs/klemming/home/q/qunli/qundir/AML/data/ArchR")
scATAC_path_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scATA/"
scATAC_output_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scATAOut/"
scRNA_RDS_dir <- "/cfs/klemming/home/q/qunli/qundir/AML/data/scRNARDS/"
MACS2_location <- "/pdc/software/other/macs2/2.2.9.1/bin/macs2"
dbArchR <- "/cfs/klemming/home/q/qunli/qundir/AML/data/ArchR/Annotations/ArchR-Hg38-v1.Anno"

group_colors <- c("B.Cell" = "#7DB18A", "CD16.Monocytes" = "#6F4A98", "CD4.Memory" = "#4C502D",
                  "CD4.Naive" = "#201C19", "CD8.Effector" = "#2A5B31", "CD8.Memory" = "#69AD46",
                  "CD8.Naive" = "#5B521A", "cDC" = "#8587B1", "CLP" = "#B9D5AB", "Early.Basophil" = "#C",
                  "Early.Erythrocyte" = "#B12579", "GMP" = "#25316E", "HSC" = "#EAA63A", "Late.Erythrocyte" = "#D2E7EB",
                  "Monocytes" = "#406DB1", "NK" = "#4C662E", "pDC" = "#62B1E1", "Plasma" = "#D2E7EB",
                  "Pre.B.Cell" = "#45948E", "Progenitor" = "#AA872C", "Unknown" = "#EAEAEA")

sample <- "AML11"
lineage_type <- "M"
traj_celltype <- "predictedGroup_Un" # predictedGroup_Un Classified_Celltype
trajectory_name <- "trajectory_M" # trajectory_M Monocle_Progenitor

tmp_out_dir <- paste0(scATAC_output_dir, sample, "/")
predifined_meta_DX <- read.delim(paste0("/cfs/klemming/home/q/qunli/qundir/AML/data/scATA/", sample, "_DX#"),
rownames(predifined_meta_DX) <- paste0(sample, "_DX#", predifined_meta_DX$Cell_Barcode)
predifined_meta_REL <- read.delim(paste0("/cfs/klemming/home/q/qunli/qundir/AML/data/scATA/", sample, "_REL#"),
rownames(predifined_meta_REL) <- paste0(sample, "_REL#", predifined_meta_REL$Cell_Barcode)
predifined_meta <- rbind(predifined_meta_DX, predifined_meta_REL)
predifined_meta$cellNames <- rownames(predifined_meta)

scATAC_data <- loadArchRProject(tmp_out_dir)

scATAC_data <- addCellColData(
  ArchRProj = scATAC_data,
  data      = predifined_meta[scATAC_data$cellNames, "Classified_Celltype"],
  cells     = scATAC_data$cellNames,
  name      = "Classified_Celltype",
  force     = TRUE
)

scATAC_data <- addMotifAnnotations(ArchRProj = scATAC_data, motifSet = "cisbp", name = "Motif", force=TRUE)
scATAC_data <- addDeviationsMatrix(ArchRProj = scATAC_data, peakAnnotation = "Motif", force = TRUE)
scATAC_data <- addArchRAnnotations(ArchRProj = scATAC_data, collection = "ATAC", db = dbArchR, force = TRUE)

cellsInSample <- grep("DX|REL", scATAC_data$cellNames, value = TRUE)
scATAC_data <- subsetArchRProject(scATAC_data, cells = cellsInSample, force = TRUE)

# trajectory analysis
trajectory_M <- c("HSC", "Progenitor", "GMP", "Monocytes", "CLP", "Pre.B.Cell", "B.Cell")

```

```

trajectory_M <- c("HSC", "Progenitor", "GMP", "Monocytes")

scATAC_data<- addTrajectory(
  ArchRProj = scATAC_data,
  name = trajectory_name,
  groupBy = traj_celltype,
  trajectory = trajectory_M,
  embedding = "UMAP",
  force = TRUE
)

#Monocle3
monocle_solo <- getMonocleTrajectories(
  ArchRProj = scATAC_data,
  name = trajectory_name,
  useGroups = trajectory_M,
  principalGroup = "HSC",
  groupBy = traj_celltype,
  embedding = "UMAP",
  clusterParams = list(k = 50),
  seed = 1
)

scATAC_data <- addMonocleTrajectory(
  ArchRProj = scATAC_data,
  name = trajectory_name,
  useGroups = trajectory_M,
  groupBy = traj_celltype,
  monocleCDS = monocle_solo,
  force = TRUE
)

scATAC_data <- addImputeWeights(scATAC_data)
geneName_Q <- "MEF2C"

#ArchR
p1_GeneScoreMatrix <- plotTrajectory(scATAC_data, trajectory = trajectory_name, colorBy = "GeneScoreMatrix")
p2_GeneIntegrationMatrix <- plotTrajectory(scATAC_data, trajectory = trajectory_name, colorBy = "GeneIntegrationMatrix")
p3_cellColData <- plotTrajectory(scATAC_data, trajectory = trajectory_name, colorBy = "cellColData", name = traj_celltype)
p4_UMAP_celltype <- plotEmbedding(ArchRProj = scATAC_data, colorBy = "cellColData", name = traj_celltype)
p5_UMAP_stage <- plotEmbedding(ArchRProj = scATAC_data, colorBy = "cellColData", name = "Sample", embedding = "UMAP")

name_p1_GeneScoreMatrix <- paste0("p1_GeneScoreMatrix_", geneName_Q, "_", sample, "_", lineage_type, ".pdf")
name_p2_GeneIntegrationMatrix <- paste0("p2_GeneIntegrationMatrix_", geneName_Q, "_", sample, "_", lineage_type, ".pdf")
name_p3_cellColData <- paste0("p3_cellColData_", sample, "_", lineage_type, ".pdf")
name_p4_UMAP_celltype <- paste0("p4_UMAP_celltype_", sample, "_", lineage_type, ".pdf")
name_p5_UMAP_stage <- paste0("p5_UMAP_stage_", sample, "_", lineage_type, ".pdf")
name_p6_gene_time <- paste0(tmp_out_dir, "Plots/", "p6_gene_time_", sample, "_", lineage_type, ".pdf")

plotPDF(p1_GeneScoreMatrix, name = name_p1_GeneScoreMatrix, ArchRProj = scATAC_data, addDOC = FALSE, width = 5,
plotPDF(p2_GeneIntegrationMatrix, name = name_p2_GeneIntegrationMatrix, ArchRProj = scATAC_data, addDOC = FALSE, width = 5,
plotPDF(p3_cellColData, name = name_p3_cellColData, ArchRProj = scATAC_data, addDOC = FALSE, width = 5,

```

```

plotPDF(p4_UMAP_celltype, name = name_p4_UMAP_celltype, ArchRProj = scATAC_data, addDOC = FALSE, width = 10, height = 8)
plotPDF(p5_UMAP_stage, name = name_p5_UMAP_stage, ArchRProj = scATAC_data, addDOC = FALSE, width = 5, height = 8)

pdf(name_p6_gene_time, width = 10, height = 8)
ggAlignPlots(p1_GeneScoreMatrix[[2]], p2_GeneIntegrationMatrix[[2]], type = "h")
dev.off()

# Pseudo-time heatmaps
trajMM <- getTrajectory(ArchRProj = scATAC_data, name = trajectory_name, useMatrix = "MotifMatrix", log = FALSE)
trajGIM <- getTrajectory(ArchRProj = scATAC_data, name = trajectory_name, useMatrix = "GeneIntegrationMatrix", log = FALSE)
corGIM_MM <- correlateTrajectories(trajGIM, trajMM)
idxToRemove2 <- grep(pattern = "deviations", x = corGIM_MM[["correlatedMappings"]])$name2)
if(length(idxToRemove2 > 1)){
  corGIM_MM[["correlatedMappings"]] <- corGIM_MM[["correlatedMappings"]][-idxToRemove2,]
}

trajGIM2 <- trajGIM[corGIM_MM[[1]]$name1, ]
trajMM2 <- trajMM[corGIM_MM[[1]]$name2, ]
trajCombined <- trajGIM2
assay(trajCombined, withDimnames=FALSE) <- t(apply(assay(trajGIM2), 1, scale)) + t(apply(assay(trajMM2), 1, scale))
combinedMat <- plotTrajectoryHeatmap(trajCombined, returnMat = TRUE, varCutOff = 0)
rowOrder <- match(rownames(combinedMat), rownames(trajGIM2))
ht1 <- plotTrajectoryHeatmap(trajGIM2, pal = paletteContinuous(set = "blueYellow", varCutOff = 0, rowOrder = rowOrder))
ht2 <- plotTrajectoryHeatmap(trajMM2, pal = paletteContinuous(set = "solarExtra", varCutOff = 0, rowOrder = rowOrder))

name_p7_heatmap <- paste0(tmp_out_dir, "Plots/", "p7_heatmap_", sample, "_", lineage_type, ".pdf")
pdf(name_p7_heatmap, width = 10, height = 8)
ComplexHeatmap::draw(ht1 + ht2)
dev.off()

# save assay data
trajGIM2_assay <- as.data.frame(t(apply(assay(trajGIM2), 1, scale)))
trajGIM2_colData <- as.data.frame(trajGIM2@colData)
trajGIM2_colData$time <- rownames(trajGIM2_colData)

trajMM2_assay <- as.data.frame(t(apply(assay(trajMM2), 1, scale)))
trajMM2_colData <- as.data.frame(trajMM2@colData)
trajMM2_colData$time <- rownames(trajMM2_colData)

trajGIM2_rds <- list(assayMat = trajGIM2_assay, assayColData = trajGIM2_colData)
trajMM2_rds <- list(assayMat = trajMM2_assay, assayColData = trajMM2_colData)

name_rds_GIM2 <- paste0(tmp_out_dir, "Plots/", "p8_RDS_GIM2_", sample, "_", lineage_type, ".rds")
name_rds_MM2 <- paste0(tmp_out_dir, "Plots/", "p8_RDS_MM2_", sample, "_", lineage_type, ".rds")
saveRDS(trajGIM2_rds, name_rds_GIM2)
saveRDS(trajMM2_rds, name_rds_MM2)

```

5 pipelines for generating single-cell somatic mutation data list

```
load_CSM <- function(csm_path, somatic_locs){
  data_CSM_raw <- read.csv(csm_path, header = T, row.names = 1)
  data_CSM_raw <- as.data.frame(t(data_CSM_raw))
  data_CSM <- data_CSM_raw %>%
    select(any_of(somatic_locs))

  data_MutPerCell <- data.frame(
    CellTag = rownames(data_CSM),
    MutNum_somatic = rowSums(data_CSM == "Mut"),
    WTNum_somatic = rowSums(data_CSM == "WT")
  )

  data_CellPerMut <- data.frame(
    Location = colnames(data_CSM),
    CellMutNum = colSums(data_CSM == "Mut"),
    CellWTNum = colSums(data_CSM == "WT")
  )

  cells_selected <- data_MutPerCell %>% filter(MutNum_somatic > 0) %>% pull(CellTag)
  locs_selected <- data_CellPerMut %>% filter(CellMutNum > 0) %>% pull(Location)
  data_CSM_clean <- data_CSM[cells_selected, locs_selected]
  rm(data_CSM_raw, data_CSM, data_MutPerCell, data_CellPerMut, cells_selected, locs_selected)
  return(data_CSM_clean)
}

# ATA
ATA_samples <- c("AML1", "AML2", "AML3", "AML4", "AML5", "AML6", "AML7", "AML8",
  "AML10", "AML11", "AML12", "AML13", "AML14", "AML15", "AML16", "AML17", "AML18", "AML19",
  "AML20", "AML21", "AML22", "AML24", "AML25", "AML26", "AML27")

AML_ATA_CSM_list <- list()

for (sampleID in ATA_samples){
  message(paste0("Processing ", sampleID))
  # generate from scMutrace pipeline
  CSM_path <- paste0("../Data/AML/Mutation/scATA/", sampleID, "/gCellSNPMatrix.step3.1_excludeControlSi")
  CSM_tmp <- load_CSM(CSM_path, AML_scATA_locs)
  AML_ATA_CSM_list[[sampleID]] <- CSM_tmp
  rm(CSM_path, CSM_tmp)
}

# RNA
RNA_samples <- c("AML1", "AML2", "AML3", "AML4", "AML5", "AML6", "AML7", "AML8", "AML9", "AML10",
  "AML11", "AML12", "AML13", "AML14", "AML15", "AML16", "AML17", "AML18", "AML19",
  "AML20", "AML21", "AML22", "AML23", "AML24", "AML25", "AML26", "AML27", "AML28")

AML_RNA_CSM_list <- list()

for (sampleID in RNA_samples){
  message(paste0("Processing ", sampleID))
  # generate from scMutrace pipeline
```

```

CSM_path <- paste0("../Data/AML/Mutation/scRNA/", sampleID, "/gCellSNPMatrix.step3.1_excludeControlSi
CSM_tmp <- load_CSM(CSM_path, AML_scRNA_locs)
AML_RNA_CSM_list[[sampleID]] <- CSM_tmp
rm(CSM_path, CSM_tmp)
}

saveRDS(AML_ATA_CSM_list, "../Data/AML/AML_ATA_CSM_list.rds")
saveRDS(AML_RNA_CSM_list, "../Data/AML/AML_RNA_CSM_list.rds")

```

6 pipelines for generating sample stage mutation similarity matrix

```

# may spent 2hrs
sample_order_keep <- c(
  "AML1_DX", "AML1_REM",
  "AML2_DX", "AML2_REM", "AML2_REL",
  "AML3_DX", "AML3_REM",
  "AML4_DX", "AML4_REL",
  "AML5_DX", "AML5_REM", "AML5_REL",
  "AML6_DX", "AML6_REL",
  "AML7_REM", "AML7_REL",
  "AML8_DX", "AML8_REM", "AML8_REL",
  "AML10_DX", "AML10_REM", "AML10_REL",
  "AML11_DX", "AML11_REM", "AML11_REL",
  "AML12_DX", "AML12_REL",
  "AML13_DX", "AML13_REL",
  "AML15_DX", "AML15_REM", "AML15_REL",
  "AML16_DX", "AML16_REM", "AML16_REL",
  "AML17_DX", "AML17_REL",
  "AML18_DX", "AML18_REL",
  "AML19_DX", "AML19_REL",
  "AML20_DX", "AML20_REM",
  "AML21_DX", "AML21_REM", "AML21_REL",
  "AML22_DX", "AML22_REM", "AML22_REL",
  "AML24_DX", "AML24_REM", "AML24_REL",
  "AML26_DX", "AML26_REM", "AML26_REL",
  "AML27_DX", "AML27_REM", "AML27_REL")

# all no filter
# rare keep freq == "rare"
# noCensus keep Tag == "noCensus"
AML_SSM_scRNA_rare_bed <- AML_SSM_scRNA_bed %>% filter(freq == "rare")
AML_SSM_scATA_rare_bed <- AML_SSM_scATA_bed %>% filter(freq == "rare")

row_names <- unique(AML_samples$scRNA)
col_names <- unique(AML_samples$scATA)

data_commonBed_similarity <- matrix(0, nrow = length(row_names), ncol = length(col_names))
rownames(data_commonBed_similarity) <- row_names
colnames(data_commonBed_similarity) <- col_names

for (i in 1:nrow(data_commonBed_similarity)){

```

```

scRNA_id_tmp <- rownames(data_commonBed_similarity)[i]
print(paste("scRNA:", scRNA_id_tmp, sep = " "))
# get sample
scRNA_col <- sym(scRNA_id_tmp)
scRNA_sample_tmp <- AML_SSM_scRNA %>%
  select(Location, !!scRNA_col) %>%
  filter(!!scRNA_col == "Mut")

for (j in 1:ncol(data_commonBed_similarity)){
  scATA_id_tmp <- colnames(data_commonBed_similarity)[j]
  print(paste("      -scATA:", scATA_id_tmp, sep = " "))
  scATA_col <- sym(scATA_id_tmp)
  scATA_sample_tmp <- AML_SSM_scATA %>%
    select(Location, !!scATA_col) %>%
    filter(!!scATA_col == "Mut")

  # common bed
  commonBed_scRNA_scATA <- bedtoolsr::bt.intersect(AML_Bed[['scRNA']] [[scRNA_id_tmp]],
                                                  AML_Bed[['scATA']] [[scATA_id_tmp]])

  # intersect
  scRNA_commonBed_tmp <- bedtoolsr::bt.intersect(AML_SSM_scRNA_rare_bed, commonBed_scRNA_scATA)$V6
  scATA_commonBed_tmp <- bedtoolsr::bt.intersect(AML_SSM_scATA_rare_bed, commonBed_scRNA_scATA)$V6

  # keep common
  scRNA_mut_tmp <- scRNA_sample_tmp %>%
    filter(Location %in% scRNA_commonBed_tmp) %>%
    pull(Location)
  scATA_mut_tmp <- scATA_sample_tmp %>%
    filter(Location %in% scATA_commonBed_tmp) %>%
    pull(Location)

  # overlap score
  common_muts <- length(unique(intersect(scRNA_mut_tmp, scATA_mut_tmp)))
  all_muts <- length(unique(union(scRNA_mut_tmp, scATA_mut_tmp)))
  score_tmp <- if (all_muts > 0) common_muts / all_muts else 0
  data_commonBed_similarity[scRNA_id_tmp, scATA_id_tmp] <- score_tmp
}
}

row_prefix <- sapply(rownames(data_commonBed_similarity), function(x) strsplit(x, "_")[[1]][1])
row_suffix <- sapply(rownames(data_commonBed_similarity), function(x) strsplit(x, "_")[[1]][2])
col_prefix <- sapply(colnames(data_commonBed_similarity), function(x) strsplit(x, "_")[[1]][1])
col_suffix <- sapply(colnames(data_commonBed_similarity), function(x) strsplit(x, "_")[[1]][2])

col_anno <- HeatmapAnnotation(
  Sample = anno_text(col_prefix, gp = gpar(fontsize = 8)),
  scATACStage = col_suffix,
  col = list(scATACStage = c("DX" = "#d9d9d9", "REM" = "#969696", "REL" = "#000000"))
)

row_anno <- rowAnnotation(
  Sample = anno_text(row_prefix, gp = gpar(fontsize = 8)),

```



```

scRNAStage = row_suffix,
col = list(scRNAStage = c("DX" = "#d9d9d9", "REM" = "#969696", "REL" = "#000000")))

data_commonBed_similarity <- data_commonBed_similarity[sample_order, sample_order]

Heatmap(data_commonBed_similarity,
  top_annotation = col_anno,
  left_annotation = row_anno,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  row_names_gp = gpar(fontsize = 6),
  column_names_gp = gpar(fontsize = 6),
  width = unit(10, "cm"), height = unit(10, "cm"),
  layer_fun = function(j, i, x, y, width, height, fill) {

    grid.rect(x = x, y = y, width = width, height = height,
      gp = gpar(col = NA, fill = fill))

    match_prefix <- row_prefix[i] == col_prefix[j]
    if (any(match_prefix)) {
      grid.rect(x = x[match_prefix], y = y[match_prefix],
        width = width, height = height,
        gp = gpar(col = "#000000", lwd = 0.1, fill = NA))
    }
  })

```

7 pipelines for generating mutgeneExp

```

# mut_id > cell_id > mut_status > celltype > geneSymbolMut > geneSymbolExp > Stage
sample_selected_scRNA <- list(
  AML1 = c("AML1_DX", "AML1_REM"),
  AML2 = c("AML2_DX", "AML2_REM", "AML2_REL"),
  AML3 = c("AML3_DX", "AML3_REM"),
  AML4 = c("AML4_DX", "AML4_REL"),
  AML5 = c("AML5_DX", "AML5_REM", "AML5_REL"),
  AML6 = c("AML6_DX", "AML6_REM", "AML6_REL"),
  AML8 = c("AML8_DX", "AML8_REM", "AML8_REL"),
  AML9 = c("AML9_DX", "AML9_REM", "AML9_REL"),
  AML10 = c("AML10_DX", "AML10_REM", "AML10_REL"),
  AML11 = c("AML11_DX", "AML11_REM", "AML11_REL"),
  AML12 = c("AML12_DX", "AML12_REM", "AML12_REL"),
  AML13 = c("AML13_DX", "AML13_REM", "AML13_REL"),
  AML14 = c("AML14_DX", "AML14_REM"),
  AML15 = c("AML15_DX", "AML15_REM", "AML15_REL"),
  AML16 = c("AML16_DX", "AML16_REM", "AML16_REL"),
  AML17 = c("AML17_DX", "AML17_REL"),
  AML18 = c("AML18_DX", "AML18_REL"),
  AML19 = c("AML19_DX", "AML19_REL"),
  AML20 = c("AML20_DX", "AML20_REM"),
  AML21 = c("AML21_DX", "AML21_REM", "AML21_REL"),
  AML22 = c("AML22_DX", "AML22_REM", "AML22_REL"),

```



```

AML23 = c("AML23_DX", "AML23_REM", "AML23_REL"),
AML24 = c("AML24_DX", "AML24_REM", "AML24_REL"),
AML25 = c("AML25_DX", "AML25_REM", "AML25_REL"),
AML26 = c("AML26_DX", "AML26_REM", "AML26_REL"),
AML27 = c("AML27_DX", "AML27_REM", "AML27_REL")
)

AML_scrNA_meta$cell_id_col <- sub("_[0-9]+$", "", gsub("[.-]", "_", rownames(AML_scrNA_meta)))
AML_scrNA_meta$cell_id_col <- gsub("#", "_", AML_scrNA_meta$cell_id_col)

AML_scrNA_mutAnn <- read.table("../Data/AML/EGA_scrNA.hg38_multianno.txt", sep = "\t", header = T)

add_meta_to_res <- function(res, meta, cell_id_col = "cell_id_col", meta_cols = "Classified_Celltype") {
  dplyr::left_join(res, meta[, c(cell_id_col, meta_cols), drop = FALSE], by = cell_id_col)
}

extrac_gene2score_bygene <- function(gene2score, gene_id){
  gene2score_gene <- as.data.frame(t(gene2score[gene_id, ]))
  gene2score_gene$cell_id_col <- gsub("#", "_", sub("-[0-9]+$", "", rownames(gene2score_gene)))
  colnames(gene2score_gene) <- c("gene_score", "cell_id_col")
  gene2score_gene$geneSymbol <- gene_id
  return(gene2score_gene)
}

data_mutgeneExp <- data.frame(
  mut_id_col = character(),
  cell_id_col_raw = character(),
  cell_id_col = character(),
  mut_status_col = character(),
  stage_col = character(),
  Classified_Celltype = character(),
  geneSymbol = character(),
  gene_score = numeric(),
  stringsAsFactors = FALSE
)

for (sample in names(sample_selected_scrNA)){
  message(paste0("Proceessiong ", sample))
  csm_sample <- AML_scrNA_CSM_list[[sample]]
  mut_sample_total <- colnames(csm_sample)
  sample_stages <- extract_sampleID(rownames(csm_sample))
  for (stage in sample_stages){
    path_geneExp <- paste0("../Data/AML/singleCellMatrix/scRNA/", stage, "_scRNA.rds")
    gene2Exp <- readRDS(path_geneExp)
    gene2Exp_df <- as.data.frame(gene2Exp@assays$RNA$counts)
    colnames(gene2Exp_df) <- paste0(stage, "_", sub("-1$", "", colnames(gene2Exp_df)))
    for (mut in mut_sample_total){
      cells_mut <- retrieveCellsByLocation(csm_sample, mut)$Mut
      cells_wt <- retrieveCellsByLocation(csm_sample, mut)$WT
      cells_mut_stage <- grep(stage, cells_mut, value = TRUE)
      cells_wt_stage <- grep(stage, cells_wt, value = TRUE)
      mut2gene <- AML_scrNA_mutAnn %>% dplyr::filter(Location == mut) %>% dplyr::pull(Gene.refGene)
      n_mut <- length(cells_mut_stage)
    }
  }
}

```

```

n_wt <- length(cells_wt_stage)
n_all <- n_mut + n_wt
res <- data.frame(
  mut_id_col = rep(mut, n_all),
  cell_id_col_raw = c(cells_mut_stage, cells_wt_stage),
  cell_id_col = gsub("_L[0-9]+_", "_", c(cells_mut_stage, cells_wt_stage)),
  mut_status_col = c(rep("Mut", n_mut), rep("WT", n_wt)),
  stage_col = rep(stage, n_all),
  stringsAsFactors = FALSE
)
# add cell type
res <- add_meta_to_res(res, AML_scrNA_meta)
# add gene exp
geneExp_gene <- extrac_gene2score_bygene(gene2Exp_df, mut2gene)
res <- add_meta_to_res(res, geneExp_gene, "cell_id_col", c("geneSymbol", "gene_score"))
data_mutgeneExp <- rbind(data_mutgeneExp, res)
}
}
}

HSPC <- c("HSC", "Progenitor")
Myeloid <- c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid <- c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive", "CD4.Memory",
             "CD8.Naive", "CD8.Memory", "CD8.Effector", "NK")
Erythrocytic <- c("Early.Erythrocyte", "Late.Erythrocyte")
Other <- c("Early.Basophil", "Plasma")

data_mutgeneExp$mainCelltype <- with(data_mutgeneExp, ifelse(Classified_Celltype %in% HSPC, "HSPC",
  ifelse(Classified_Celltype %in% Myeloid, "Myeloid",
  ifelse(Classified_Celltype %in% Lymphoid, "Lymphoid",
  ifelse(Classified_Celltype %in% Erythrocytic, "Erythrocytic",
  ifelse(Classified_Celltype %in% Other, "Other", "Unknown"))))))

colnames(data_mutgeneExp) <- c("mut_id_col", "cell_id_col_raw", "cell_id_col",
  "mut_status_col", "stage_id_col", "Classified_Celltype",
  "geneSymbolMut", "gene_scoreExp", "mainCelltype")

saveRDS(data_mutgeneExp, "../Results/AML/data_AML_scrNA_mutgeneExp.rds")

```

8 pipelines for generating mutpeakgenedata

```

# mut_id > peak_id > cell_id > mut_status > celltype > peak_fragment_count > peak2gene_symbol > peak2g
sample_selected_scATA <- list(
  AML1 = c("AML1_DX", "AML1_REM"),
  AML2 = c("AML2_DX", "AML2_REM", "AML2_REL"),
  AML3 = c("AML3_DX", "AML3_REM"),
  AML4 = c("AML4_DX", "AML4_REL"),
  AML5 = c("AML5_DX", "AML5_REM", "AML5_REL"),
  AML6 = c("AML6_DX", "AML6_REL"),
  AML8 = c("AML8_DX", "AML8_REM", "AML8_REL"),
  AML10 = c("AML10_DX", "AML10_REM", "AML10_REL"),

```

```

AML11 = c("AML11_DX", "AML11_REM", "AML11_REL"),
AML12 = c("AML12_DX", "AML12_REL"),
AML13 = c("AML13_DX", "AML13_REL"),
AML15 = c("AML15_DX", "AML15_REM", "AML15_REL"),
AML16 = c("AML16_DX", "AML16_REM", "AML16_REL"),
AML17 = c("AML17_DX", "AML17_REL"),
AML18 = c("AML18_DX", "AML18_REL"),
AML19 = c("AML19_DX", "AML19_REL"),
AML20 = c("AML20_DX", "AML20_REM"),
AML21 = c("AML21_DX", "AML21_REM", "AML21_REL"),
AML22 = c("AML22_DX", "AML22_REL"),
AML24 = c("AML24_DX", "AML24_REM", "AML24_REL"),
AML25 = c("AML25_DX", "AML25_REM", "AML25_REL"),
AML26 = c("AML26_DX", "AML26_REM", "AML26_REL"),
AML27 = c("AML27_DX", "AML27_REM", "AML27_REL")
)

AML_scATA_meta$cell_id_col <- sub("[0-9]+$", "", gsub("[.]", "_", rownames(AML_scATA_meta)))
AML_scATA_meta$cell_id_col <- gsub("#", "_", AML_scATA_meta$cell_id_col)

add_meta_to_res <- function(res, meta, cell_id_col = "cell_id_col", meta_cols = "Classified_Celltype") {
  dplyr::left_join(res, meta[, c(cell_id_col, meta_cols), drop = FALSE], by = cell_id_col)
}

extrac_peak2cell_bypeak <- function(peak2cell, peak_id){
  peak2cell_peak <- as.data.frame(t(peak2cell[peak_id,]))
  peak2cell_peak$cell_id_col <- gsub("#", "_", sub("-[0-9]+$", "", rownames(peak2cell_peak)))
  colnames(peak2cell_peak) <- c("fragment_count", "cell_id_col")
  return(peak2cell_peak)
}

extrac_gene2score_bygene <- function(gene2score, gene_id){
  gene2score_gene <- as.data.frame(t(gene2score[gene_id, ]))
  gene2score_gene$cell_id_col <- gsub("#", "_", sub("-[0-9]+$", "", rownames(gene2score_gene)))
  colnames(gene2score_gene) <- c("gene_score", "cell_id_col")
  gene2score_gene$geneSymbol <- gene_id
  return(gene2score_gene)
}

data_mutpeakgene <- data.frame(
  cell_id_col = character(),
  mut_id_col = character(),
  peak_id_col = character(),
  mut_status_col = character(),
  Classified_Celltype = character(),
  fragment_count = numeric(),
  geneSymbol = character(),
  gene_score = numeric(),
  sample_id_col = character(),
  stage_id_col = character(),
  stringsAsFactors = FALSE
)

```

```

for (sample in names(sample_selected_scATA)){
  message(paste0("Processing ", sample))
  csm_sample <- AML_scATA_CSM_list[[sample]]
  mut_sample_total <- colnames(csm_sample)
  path_peak2cell <- paste0("../Data/AML/singleCellMatrix/scATAC_peak2cells/", sample, "_peakCell.rds")
  path_peak2gene <- paste0("../Data/AML/singleCellMatrix/scATAC_peak2Genes/", sample, "_peak2Genes.rds")
  path_gene2scor <- paste0("../Data/AML/singleCellMatrix/scATAC_geneExp/", sample, "_geneExp.rds")
  peak2Cell <- readRDS(path_peak2cell)
  peak2Gene <- readRDS(path_peak2gene)
  gene2Scor <- readRDS(path_gene2scor)
  peak2Gene <- peak2Gene %>% dplyr::filter(Correlation >= 0.4 & FDR <= 0.05)
  df_peak_mut <- process_mut_peak(rownames(peak2Cell), mut_sample_total)
  for (i_row in 1:nrow(df_peak_mut)){
    mut_id <- df_peak_mut[i_row,]$mut_id
    peak_id <- df_peak_mut[i_row,]$peak_id
    peak2Cell_peakid <- extrac_peak2cell_bypeak(peak2Cell, peak_id)
    peak2Gene_peakid <- peak2Gene %>% dplyr::filter(peakName == gsub("[:\\-]", "_", peak_id)) %>% dplyr::
    if (length(peak2Gene_peakid) == 0) {next}
    mut_cells_all <- retrieveCellsByLocation(csm_sample, mut_id)$Mut
    wt_cells_all <- retrieveCellsByLocation(csm_sample, mut_id)$WT
    n_mut <- length(mut_cells_all)
    n_wt <- length(wt_cells_all)
    n_all <- n_mut + n_wt
    res <- data.frame(
      mut_id_col = rep(mut_id, n_all),
      peak_id_col = rep(peak_id, n_all),
      cell_id_col = c(mut_cells_all, wt_cells_all),
      mut_status_col = c(rep("Mut", n_mut), rep("WT", n_wt)),
      stringsAsFactors = FALSE
    )
    # add cell type
    res <- add_meta_to_res(res, AML_scATA_meta)
    # add fragment count
    res <- add_meta_to_res(res, peak2Cell_peakid, "cell_id_col", "fragment_count")
    # add gene score
    res_genescore_list <- list()
    for (gene in peak2Gene_peakid){
      genescore_gene <- extrac_gene2score_bygene(gene2Scor, gene)
      res_tmp <- add_meta_to_res(res, genescore_gene, "cell_id_col", c("geneSymbol", "gene_score"))
      res_genescore_list[[gene]] <- res_tmp
    }
    res_genescore <- do.call(rbind, res_genescore_list)
    res_genescore$sample_id_col <- sample
    res_genescore$stage_id_col <- sub("^([A-Za-z0-9]+_[A-Za-z0-9]+).*$", "\\1", res_genescore$cell_id_col)
    data_mutpeakgene <- rbind(data_mutpeakgene, res_genescore)
  }
}

rownames(data_mutpeakgene) <- NULL

saveRDS(data_mutpeakgene, "../Results/AML/data_AML_scATA_mutpeakgene.rds")

```

9 pipelines for stage diff muts

```
run_fisher_stage <- function(df) {  
  
  # pivot_wider generate wide format mut_status_col  
  df_wide <- df %>%  
    tidyr::pivot_wider(  
      names_from = mut_status_col,  
      values_from = cell_count,  
      values_fill = 0  
    )  
  
  # exclude stage_id_col and mut_id_col  
  status_cols <- setdiff(colnames(df_wide), c("stage_id_col", "mut_id_col"))  
  
  # add missing column if only one status present  
  if (length(status_cols) == 1) {  
    missing_col <- ifelse(status_cols == "Mut", "WT", "Mut")  
    df_wide[[missing_col]] <- 0  
    status_cols <- c(status_cols, missing_col)  
  }  
  
  # rename status columns to "Mut" and "WT"  
  names(df_wide)[match(status_cols, names(df_wide))] <- c("Mut", "WT")  
  
  stages <- unique(df_wide$stage_id_col)  
  
  if (length(stages) < 2) {  
    return(tibble(  
      stage1 = character(0),  
      stage2 = character(0),  
      p.value = numeric(0),  
      log2_OR = numeric(0),  
      estimate = numeric(0),  
      table = character(0)  
    ))  
  }  
  
  purrr::map_df(combn(stages, 2, simplify = FALSE), function(stg) {  
  
    d1 <- dplyr::filter(df_wide, stage_id_col == stg[1])  
    d2 <- dplyr::filter(df_wide, stage_id_col == stg[2])  
  
    mut1 <- ifelse(length(d1$Mut) == 0, 0, d1$Mut)  
    wt1 <- ifelse(length(d1$WT) == 0, 0, d1$WT)  
    mut2 <- ifelse(length(d2$Mut) == 0, 0, d2$Mut)  
    wt2 <- ifelse(length(d2$WT) == 0, 0, d2$WT)  
  
    frac1 <- ifelse((mut1 + wt1) == 0, NA, mut1 / (mut1 + wt1))  
    frac2 <- ifelse((mut2 + wt2) == 0, NA, mut2 / (mut2 + wt2))  
  
    tab <- matrix(c(mut1, wt1, mut2, wt2), nrow = 2, byrow = TRUE)
```

```

# if all zero, return NA
if (sum(tab) == 0) {
  return(tibble(
    stage1 = stg[1],
    stage2 = stg[2],
    p.value = NA,
    log2_OR = NA,
    estimate = NA,
    frac_stage1 = NA,
    frac_stage2 = NA,
    table = paste(tab, collapse = ",")
  ))
}

ft <- fisher.test(tab)

tibble(
  stage1 = stg[1],
  stage2 = stg[2],
  p.value = ft$p.value,
  log2_OR = log2(ft$estimate),
  estimate = ft$estimate,
  frac_stage1 = frac1,
  frac_stage2 = frac2,
  table = paste(tab, collapse = ",")
)
})
}

AML_scRNA_stageMutCount <- AML_RNA_mutExprGene %>%
  dplyr::group_by(mut_id_col, stage_id_col, mut_status_col, mainCelltype) %>%
  dplyr::summarise(cell_count = n(), .groups = "drop")

AML_scATA_stageMutCount <- AML_ATA_mutPeakGene %>%
  dplyr::group_by(mut_id_col, stage_id_col, mut_status_col, mainCelltype) %>%
  dplyr::summarise(cell_count = n(), .groups = "drop")

res_scRNA <- AML_scRNA_stageMutCount %>%
  dplyr::group_by(mut_id_col, mainCelltype) %>%
  dplyr::group_modify(~ run_fisher_stage(.x)) %>%
  dplyr::ungroup() %>%
  dplyr::mutate(
    Tag = paste0(
      gsub(".*_", "", stage1),
      "_vs_",
      gsub(".*_", "", stage2)
    )
  )

res_scATA <- AML_scATA_stageMutCount %>%
  dplyr::group_by(mut_id_col, mainCelltype) %>%
  dplyr::group_modify(~ run_fisher_stage(.x)) %>%
  dplyr::ungroup() %>%

```

```

dplyr::mutate(
  Tag = paste0(
    gsub(".*_", "", stage1),
    "_vs_",
    gsub(".*_", "", stage2)
  )
)

```

10 pipelines for generating celltype similarity (scRNA vs scATAC)

```

data_mutationCelltype_similarity_scRNA_scATAC <- list()
common_sample_pairs <- unique(intersect(names(AML_scATA_CSM_list),
                                         names(AML_scRNA_CSM_list)))

ct_forComparision <- setdiff(cellTypes, "Unknown")

for (sample in common_sample_pairs){
  message(sample)
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparision),
                                                    ncol = length(ct_forComparision)))

  rownames(mut_cellTypeAnn_similarity) <- ct_forComparision
  colnames(mut_cellTypeAnn_similarity) <- ct_forComparision
  csm_scRNA <- AML_scRNA_CSM_list[[sample]]
  csm_scATA <- AML_scATA_CSM_list[[sample]]

  for (i in 1:nrow(mut_cellTypeAnn_similarity)){
    # scRNA
    ct_scRNA <- rownames(mut_cellTypeAnn_similarity)[i]
    cell_barcode_scRNA_meta <- AML_scRNA_meta %>%
      mutate(CellTag = rownames(.)) %>%
      dplyr::filter(Classified_Celltype == ct_scRNA & Lambo_et_al_ID == sample) %>%
      pull(CellTag)
    cell_barcode_scRNA_csm_format_all <- rownames(csm_scRNA) %>%
      sub("_L[0-9]+_", "_", .) %>%
      sub("_(.*)_", "_\\1.", .) %>%
      paste0(., "-1") %>%
      sub("\\.", "#", x = .)

    cell_barcode_scRNA_csm_selected <- rownames(csm_scRNA)[cell_barcode_scRNA_csm_format_all %in% cell_barcode_scRNA_csm_selected]
    mut_site_scRNA <- retrieveLocsByCells(csm_scRNA, cell_barcode_scRNA_csm_selected)$Mut

    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      # scATA
      ct_scATA <- colnames(mut_cellTypeAnn_similarity)[j]
      cell_barcode_scATA_meta <- AML_scATA_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(predictedGroup_Un == ct_scATA & SampleID == sample) %>%
        pull(CellTag)
      cell_barcode_scATA_csm_format_all <- rownames(csm_scATA) %>%
        sub("_L[0-9]+_", "_", .) %>%

```

```

sub("_(.*)_", "\\1.", .) %>%
paste0(., "-1") %>%
sub("\\.", "#", x = .)

cell_barcode_scATA_csm_selected <- rownames(csm_scATA)[cell_barcode_scATA_csm_format_all %in% cell_barcode_scATA_csm_selected]
mut_site_scATA <- retrieveLocsByCells(csm_scATA, cell_barcode_scATA_csm_selected)$Mut
similarity_score <- ifelse(
  length(unique(c(mut_site_scRNA, mut_site_scATA))) == 0,
  0,
  length(unique(intersect(mut_site_scRNA, mut_site_scATA))) /
    length(unique(c(mut_site_scRNA, mut_site_scATA)))
)
mut_cellTypeAnn_similarity[i, j] <- similarity_score
}
}

data_mutationCelltype_similarity_scRNA_scATAC[[sample]] <- mut_cellTypeAnn_similarity
}

#data_mutationCelltype_similarity_scRNA_scATAC <- readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA_scATAC.rds")
similarity_sum_scRNA_scATAC <- Reduce("+", data_mutationCelltype_similarity_scRNA_scATAC)
similarity_mean_scRNA_scATAC <- similarity_sum_scRNA_scATAC / length(data_mutationCelltype_similarity_scRNA_scATAC)

breaks <- seq(min(as.matrix(similarity_mean_scRNA_scATAC)),
               max(as.matrix(similarity_mean_scRNA_scATAC)),
               length.out = 4)

col_fun <- colorRamp2(
  breaks = breaks,
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)

p_cellType_similarity_Mutation <- Heatmap(
  as.matrix(similarity_mean_scRNA_scATAC),
  name = "Similarity",
  col = col_fun,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  show_row_names = TRUE,
  show_column_names = TRUE,
  row_names_gp = gpar(fontsize = 8),
  column_names_gp = gpar(fontsize = 8),
  row_title = NULL,
  column_title = NULL,
  top_annotation = col_ha,
  left_annotation = row_ha,
  rect_gp = gpar(col = "white", lwd = 1)
)

p_cellType_similarity_scRNAscATAC_Mutation

```


11 pipelines for generating celltype similarity (scRNA)

```

data_mutationCelltype_similarity_scRNA <- list()
common_sample_pairs <- unique(intersect(names(AML_scATA_CSM_list),
                                         names(AML_scRNA_CSM_list)))

ct_forComparision <- setdiff(cellTypes, "Unknown")

for (sample in common_sample_pairs){
  message(sample)
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparision),
                                                    ncol = length(ct_forComparision)))

  rownames(mut_cellTypeAnn_similarity) <- ct_forComparision
  colnames(mut_cellTypeAnn_similarity) <- ct_forComparision
  csm_scRNA <- AML_scRNA_CSM_list[[sample]]

  for (i in 1:nrow(mut_cellTypeAnn_similarity)){
    # scRNA
    ct_scRNA_i <- rownames(mut_cellTypeAnn_similarity)[i]
    cell_barcode_scRNA_meta_i <- AML_scRNA_meta %>%
      mutate(CellTag = rownames(.)) %>%
      dplyr::filter(Classified_Celltype == ct_scRNA_i & Lambo_et_al_ID == sample) %>%
      pull(CellTag)
    cell_barcode_scRNA_csm_format_all_i <- rownames(csm_scRNA) %>%
      sub("_L[0-9]+_", "_", .) %>%
      sub("_(.*)_ ", "_\\1.", .) %>%
      paste0(., "-1")
    cell_barcode_scRNA_csm_selected_i <- rownames(csm_scRNA)[cell_barcode_scRNA_csm_format_all_i %in% c
    mut_site_scRNA_i <- retrieveLocsByCells(csm_scRNA, cell_barcode_scRNA_csm_selected_i)$Mut

    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      # scRNA
      ct_scRNA_j <- rownames(mut_cellTypeAnn_similarity)[j]
      cell_barcode_scRNA_meta_j <- AML_scRNA_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(Classified_Celltype == ct_scRNA_j & Lambo_et_al_ID == sample) %>%
        pull(CellTag)
      cell_barcode_scRNA_csm_format_all_j <- rownames(csm_scRNA) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)_ ", "_\\1.", .) %>%
        paste0(., "-1")
      cell_barcode_scRNA_csm_selected_j <- rownames(csm_scRNA)[cell_barcode_scRNA_csm_format_all_j %in% c
      mut_site_scRNA_j <- retrieveLocsByCells(csm_scRNA, cell_barcode_scRNA_csm_selected_j)$Mut
      similarity_score <- ifelse(
        length(unique(c(mut_site_scRNA_i, mut_site_scRNA_j))) == 0,
        0,
        length(unique(intersect(mut_site_scRNA_i, mut_site_scRNA_j))) /
          length(unique(c(mut_site_scRNA_i, mut_site_scRNA_j)))
      )
      mut_cellTypeAnn_similarity[i, j] <- similarity_score
    }
  }
}

```

```
data_mutationCelltype_similarity_scRNA[[sample]] <- mut_cellTypeAnn_similarity
}
```

12 pipelines for generating celltype similarity (scATAC)

```
data_mutationCelltype_similarity_scATA <- list()
common_sample_pairs <- unique(intersect(names(AML_scATA_CSM_list),
                                         names(AML_scRNA_CSM_list)))

ct_forComparision <- setdiff(cellTypes, "Unknown")

for (sample in common_sample_pairs){
  message(sample)
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparision),
                                                    ncol = length(ct_forComparision)))

  rownames(mut_cellTypeAnn_similarity) <- ct_forComparision
  colnames(mut_cellTypeAnn_similarity) <- ct_forComparision
  csm_scATA <- AML_scATA_CSM_list[[sample]]

  for (i in 1:nrow(mut_cellTypeAnn_similarity)){
    # scATA
    ct_scATA_i <- rownames(mut_cellTypeAnn_similarity)[i]
    cell_barcode_scATA_meta_i <- AML_scATA_meta %>%
      mutate(CellTag = rownames(.)) %>%
      dplyr::filter(predictedGroup_Un == ct_scATA_i & Sample == sample) %>%
      pull(CellTag)
    cell_barcode_scATA_csm_format_all_i <- rownames(csm_scATA) %>%
      sub("_L[0-9]+_", "_", .) %>%
      sub("_(.*)_ ", "\\1.", .) %>%
      paste0(., "-1") %>%
      sub("\\.", "#", x = .)
    cell_barcode_scATA_csm_selected_i <- rownames(csm_scATA)[cell_barcode_scATA_csm_format_all_i %in% c
    mut_site_scATA_i <- retrieveLocsByCells(csm_scATA, cell_barcode_scATA_csm_selected_i)$Mut

    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      # scATA
      ct_scATA_j <- colnames(mut_cellTypeAnn_similarity)[j]
      cell_barcode_scATA_meta_j <- AML_scATA_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(predictedGroup_Un == ct_scATA_i & Sample == sample) %>%
        pull(CellTag)
      cell_barcode_scATA_csm_format_all_j <- rownames(csm_scATA) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)_ ", "\\1.", .) %>%
        paste0(., "-1") %>%
        sub("\\.", "#", x = .)
      cell_barcode_scATA_csm_selected_j <- rownames(csm_scATA)[cell_barcode_scATA_csm_format_all_j %in% c
      mut_site_scATA_j <- retrieveLocsByCells(csm_scATA, cell_barcode_scATA_csm_selected_j)$Mut
      similarity_score <- ifelse(
        length(unique(c(mut_site_scATA_i, mut_site_scATA_j))) == 0,
        0,
```

```

        length(unique(intersect(mut_site_scATA_i, mut_site_scATA_j))) /
        length(unique(c(mut_site_scATA_i, mut_site_scATA_j)))
    )
    mut_cellTypeAnn_similarity[i, j] <- similarity_score
  }
}

data_mutationCelltype_similarity_scATA[[sample]] <- mut_cellTypeAnn_similarity
}

```

13 pipelines for generating celltype similarity by stages(scRNA)

```

# AML_CSM_DX_list
AML_scRNA_CSM_DX_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_DX_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_DX_list)

# AML_CSM_REM_list
AML_scRNA_CSM_REM_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REM_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_REM_list)

# AML_CSM_REL_list
AML_scRNA_CSM_REL_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REL_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_REL_list)

# function
cal_Similarity <- function(common_sample_pairs, ct_forComparison, CSM_list, AML_meta){
  data_mutationCelltype_similarity <- list()
  for (sample in common_sample_pairs){
    message(sample)
    mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparison),
                                                       ncol = length(ct_forComparison)))

    rownames(mut_cellTypeAnn_similarity) <- ct_forComparison
    colnames(mut_cellTypeAnn_similarity) <- ct_forComparison
    csm_tmp <- CSM_list[[sample]]

    for (i in 1:nrow(mut_cellTypeAnn_similarity)){
      # scRNA
      ct_i <- rownames(mut_cellTypeAnn_similarity)[i]
      cell_barcode_meta_i <- AML_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(Classified_Celltype == ct_i & Lambo_et_al_ID == sample) %>%
        pull(CellTag)
      # AML1_REM#GTTCTATTCATCACCC-1
      cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
        sub("_L[0-9]+_", "_", .) %>%

```

```

      sub("_(.*)__", "\\1.", .) %>%
      paste0(., "-1") %>%
      sub("\\.", "#", x = .)
    cell_barcode_csm_selected_i <- rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_csm_selected_i]
    mut_site_i <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_i)$Mut

    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      # scRNA
      ct_j <- rownames(mut_cellTypeAnn_similarity)[j]
      cell_barcode_meta_j <- AML_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(Classified_Celltype == ct_j & Lambo_et_al_ID == sample) %>%
        pull(CellTag)
      cell_barcode_csm_format_all_j <- rownames(csm_tmp) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)__", "\\1.", .) %>%
        paste0(., "-1") %>%
        sub("\\.", "#", x = .)
      cell_barcode_csm_selected_j <- rownames(csm_tmp)[cell_barcode_csm_format_all_j %in% cell_barcode_csm_selected_j]
      mut_site_j <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_j)$Mut
      similarity_score <- ifelse(
        length(unique(c(mut_site_i, mut_site_j))) == 0,
        0,
        length(unique(intersect(mut_site_i, mut_site_j))) /
          length(unique(c(mut_site_i, mut_site_j)))
      )
      if (length(cell_barcode_meta_i) <= 5 | length(cell_barcode_meta_j) <= 5){
        message("Low cell number for ", ct_i, " or ", ct_j, " in ", sample, ": ",
          length(cell_barcode_meta_i), " or ", length(cell_barcode_meta_j))
        similarity_score <- 0
      }
      mut_cellTypeAnn_similarity[i, j] <- similarity_score
    }
  }

  data_mutationCelltype_similarity[[sample]] <- mut_cellTypeAnn_similarity
}
return(data_mutationCelltype_similarity)
}

data_mutationCelltype_similarity_scRNA_DX <- cal_Similarity(common_sample_pairs = unique(names(AML_scRNA_CSM_DX_list)),
  CSM_list = AML_scRNA_CSM_DX_list, AML_meta = AML_meta)

data_mutationCelltype_similarity_scRNA_REM <- cal_Similarity(common_sample_pairs = unique(names(AML_scRNA_CSM_REM_list)),
  CSM_list = AML_scRNA_CSM_REM_list, AML_meta = AML_meta)

data_mutationCelltype_similarity_scRNA_REL <- cal_Similarity(common_sample_pairs = unique(names(AML_scRNA_CSM_REL_list)),
  CSM_list = AML_scRNA_CSM_REL_list, AML_meta = AML_meta)

```

14 pipelines for generating celltype similarity by stages(scATA)

```
# AML_CSM_DX_list
AML_scATA_CSM_DX_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_DX_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_DX_list)

# AML_CSM_REM_list
AML_scATA_CSM_REM_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REM_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_REM_list)

# AML_CSM_REL_list
AML_scATA_CSM_REL_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REL_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_REL_list)

cal_Similarity <- function(common_sample_pairs, ct_forComparision, CSM_list, AML_meta){
  data_mutationCelltype_similarity <- list()
  for (sample in common_sample_pairs){
    message(sample)
    mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparision),
                                                       ncol = length(ct_forComparision)))

    rownames(mut_cellTypeAnn_similarity) <- ct_forComparision
    colnames(mut_cellTypeAnn_similarity) <- ct_forComparision
    csm_tmp <- CSM_list[[sample]]

    for (i in 1:nrow(mut_cellTypeAnn_similarity)){
      # scATA
      ct_i <- rownames(mut_cellTypeAnn_similarity)[i]
      cell_barcode_meta_i <- AML_meta %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(predictedGroup_Un == ct_i & SampleID == sample) %>%
        pull(CellTag)
      # AML1_REM#CCCGTTAGTTGCCTGG-1
      cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)_ ", "\\1.", .) %>%
        paste0(., "-1") %>%
        sub("\\.", "#", x = .)
      cell_barcode_csm_selected_i <- rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_meta_i]
      mut_site_i <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_i)$Mut

      for (j in 1:ncol(mut_cellTypeAnn_similarity)){
        # scATA
        ct_j <- rownames(mut_cellTypeAnn_similarity)[j]
        cell_barcode_meta_j <- AML_meta %>%
          mutate(CellTag = rownames(.)) %>%
          dplyr::filter(predictedGroup_Un == ct_j & SampleID == sample) %>%
          pull(CellTag)
```

```

cell_barcode_csm_format_all_j <- rownames(csm_tmp) %>%
  sub("_L[0-9]+_", "_", .) %>%
  sub("_(.*)_", "_\\1.", .) %>%
  paste0(., "-1") %>%
  sub("\\.", "#", x = .)
cell_barcode_csm_selected_j <- rownames(csm_tmp)[cell_barcode_csm_format_all_j %in% cell_barcode_csm_selected_j]
mut_site_j <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_j)$Mut
similarity_score <- ifelse(
  length(unique(c(mut_site_i, mut_site_j))) == 0,
  0,
  length(unique(intersect(mut_site_i, mut_site_j))) /
    length(unique(c(mut_site_i, mut_site_j)))
)

if (length(cell_barcode_meta_i) <= 5 | length(cell_barcode_meta_j) <= 5){
  message("Low cell number for ", ct_i, " or ", ct_j, " in ", sample, ":",
    length(cell_barcode_meta_i), " or ", length(cell_barcode_meta_j))
  similarity_score <- 0
}

mut_cellTypeAnn_similarity[i, j] <- similarity_score
}
}

data_mutationCelltype_similarity[[sample]] <- mut_cellTypeAnn_similarity
}
return(data_mutationCelltype_similarity)
}

data_mutationCelltype_similarity_scATA_DX <- cal_Similarity(common_sample_pairs = unique(names(AML_scATA_CSM_list)),
  CSM_list = AML_scATA_CSM_DX_list, AML_meta = AML_meta_DX)

data_mutationCelltype_similarity_scATA_REM <- cal_Similarity(common_sample_pairs = unique(names(AML_scATA_CSM_list)),
  CSM_list = AML_scATA_CSM_REM_list, AML_meta = AML_meta_REM)

data_mutationCelltype_similarity_scATA_REL <- cal_Similarity(common_sample_pairs = unique(names(AML_scATA_CSM_list)),
  CSM_list = AML_scATA_CSM_REL_list, AML_meta = AML_meta_REL)

```

15 pipelines for generating mutation similarity between two stages

```

cal_Similarity_scRNA_DX_REL <- function(common_sample_pairs, ct_forComparison, CSM_list_RNA_DX, CSM_list_RNA_REL){
  data_mutationCelltype_similarity <- list()
  for (sample in common_sample_pairs){
    message(sample)
    mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparison),
      ncol = length(ct_forComparison)))
    rownames(mut_cellTypeAnn_similarity) <- ct_forComparison
    colnames(mut_cellTypeAnn_similarity) <- ct_forComparison
    csm_tmp_RNA_DX <- CSM_list_RNA_DX[[sample]]
  }
}

```

```

csm_tmp_RNA_REL <- CSM_list_RNA_REL[[sample]]

for (i in 1:nrow(mut_cellTypeAnn_similarity)){
  # scRNA_DX
  ct_i <- rownames(mut_cellTypeAnn_similarity)[i]
  cell_barcode_meta_i <- AML_meta_RNA_DX %>%
    mutate(CellTag = rownames(.)) %>%
    dplyr::filter(Classified_Celltype == ct_i & Lambo_et_al_ID == sample) %>%
    pull(CellTag)
  # cell_barcode_meta_i AML4_DX#CACGGGTTTCGATACAC-1
  cell_barcode_csm_format_all_i <- rownames(csm_tmp_RNA_DX) %>%
    sub("_L[0-9]+_", "_", .) %>%
    sub("_(.*)_", "_\\1.", .) %>%
    paste0(., "-1") %>%
    sub("\\.", "#", x = .)
  # cell_barcode_csm_format_all_i AML17_DX#AGTGACTTCAACACCA-1
  cell_barcode_csm_selected_i <- rownames(csm_tmp_RNA_DX)[cell_barcode_csm_format_all_i %in% cell_barcode_csm_selected_i]
  # cell_barcode_csm_selected_i AML18_DX_CTCCAACGTACGATGG
  mut_site_i <- retrieveLocsByCells(csm_tmp_RNA_DX, cell_barcode_csm_selected_i)$Mut
  # mut_site_i 11_45953616_A_G
  for (j in 1:ncol(mut_cellTypeAnn_similarity)){
    # scRNA_REL
    ct_j <- rownames(mut_cellTypeAnn_similarity)[j]
    cell_barcode_meta_j <- AML_meta_RNA_REL %>%
      mutate(CellTag = rownames(.)) %>%
      dplyr::filter(Classified_Celltype == ct_j & Lambo_et_al_ID == sample) %>%
      pull(CellTag)
    cell_barcode_csm_format_all_j <- rownames(csm_tmp_RNA_REL) %>%
      sub("_L[0-9]+_", "_", .) %>%
      sub("_(.*)_", "_\\1.", .) %>%
      paste0(., "-1") %>%
      sub("\\.", "#", x = .)
    cell_barcode_csm_selected_j <- rownames(csm_tmp_RNA_REL)[cell_barcode_csm_format_all_j %in% cell_barcode_csm_selected_j]
    mut_site_j <- retrieveLocsByCells(csm_tmp_RNA_REL, cell_barcode_csm_selected_j)$Mut
    similarity_score <- ifelse(
      length(unique(c(mut_site_i, mut_site_j))) == 0,
      0,
      length(unique(intersect(mut_site_i, mut_site_j))) / length(unique(c(mut_site_i, mut_site_j)))
    )
    mut_cellTypeAnn_similarity[i, j] <- similarity_score
  }
}

data_mutationCelltype_similarity[[sample]] <- mut_cellTypeAnn_similarity
}
return(data_mutationCelltype_similarity)
}

cal_Similarity_scATA_DX_REL <- function(common_sample_pairs, ct_forComparision, CSM_list_ATA_DX, CSM_list_ATA_REL){
  data_mutationCelltype_similarity <- list()
  for (sample in common_sample_pairs){
    message(sample)
    mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(ct_forComparision),

```

```

ncol = length(ct_forComparision)))

rownames(mut_cellTypeAnn_similarity) <- ct_forComparision
colnames(mut_cellTypeAnn_similarity) <- ct_forComparision
csm_tmp_ATA_DX <- CSM_list_ATA_DX[[sample]]
csm_tmp_ATA_REL <- CSM_list_ATA_REL[[sample]]

for (i in 1:nrow(mut_cellTypeAnn_similarity)){
  # scATA_DX
  ct_i <- rownames(mut_cellTypeAnn_similarity)[i]
  cell_barcode_meta_i <- AML_meta_ATA_DX %>%
    mutate(CellTag = rownames(.)) %>%
    dplyr::filter(Classified_Celltype == ct_i & SampleID == sample) %>%
    pull(CellTag)
  cell_barcode_csm_format_all_i <- rownames(csm_tmp_ATA_DX) %>%
    sub("_L[0-9]+_", "_", .) %>%
    sub("_(.*)__", "_\\1.", .) %>%
    paste0(., "-1") %>%
    sub("\\.", "#", x = .)
  cell_barcode_csm_selected_i <- rownames(csm_tmp_ATA_DX)[cell_barcode_csm_format_all_i %in% cell_barcode_meta_i]
  mut_site_i <- retrieveLocsByCells(csm_tmp_ATA_DX, cell_barcode_csm_selected_i)$Mut

  for (j in 1:ncol(mut_cellTypeAnn_similarity)){
    # scATA
    ct_j <- rownames(mut_cellTypeAnn_similarity)[j]
    cell_barcode_meta_j <- AML_meta_ATA_REL %>%
      mutate(CellTag = rownames(.)) %>%
      dplyr::filter(Classified_Celltype == ct_j & SampleID == sample) %>%
      pull(CellTag)
    cell_barcode_csm_format_all_j <- rownames(csm_tmp_ATA_REL) %>%
      sub("_L[0-9]+_", "_", .) %>%
      sub("_(.*)__", "_\\1.", .) %>%
      paste0(., "-1") %>%
      sub("\\.", "#", x = .)
    cell_barcode_csm_selected_j <- rownames(csm_tmp_ATA_REL)[cell_barcode_csm_format_all_j %in% cell_barcode_meta_j]
    mut_site_j <- retrieveLocsByCells(csm_tmp_ATA_REL, cell_barcode_csm_selected_j)$Mut
    similarity_score <- ifelse(
      length(unique(c(mut_site_i, mut_site_j))) == 0,
      0,
      length(unique(intersect(mut_site_i, mut_site_j))) / length(unique(c(mut_site_i, mut_site_j)))
      #length(unique(intersect(mut_site_i, mut_site_j))) / min(length(unique(mut_site_i)), length(unique(mut_site_j)))
    )
    mut_cellTypeAnn_similarity[i, j] <- similarity_score
  }
}

data_mutationCelltype_similarity[[sample]] <- mut_cellTypeAnn_similarity
}
return(data_mutationCelltype_similarity)
}

HSPC <- c("HSC", "Progenitor")
Myeloid <- c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid <- c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive", "CD4.Memory", "CD8.Naive", "CD8.Memory", "CD8")

```



```

Erythroid_Basophil <- c("Early.Erythrocyte", "Late.Erythrocyte")
Other <- c("Early.Basophil", "Plasma")

AML_scRNA_meta <- AML_scRNA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      Classified_Celltype %in% HSPC ~ "HSPC",
      Classified_Celltype %in% Myeloid ~ "Myeloid",
      Classified_Celltype %in% Lymphoid ~ "Lymphoid",
      Classified_Celltype %in% Erythroid_Basophil ~ "Erythroid",
      Classified_Celltype %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta <- AML_scATA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      predictedGroup_Un %in% HSPC ~ "HSPC",
      predictedGroup_Un %in% Myeloid ~ "Myeloid",
      predictedGroup_Un %in% Lymphoid ~ "Lymphoid",
      predictedGroup_Un %in% Erythroid_Basophil ~ "Erythroid",
      predictedGroup_Un %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta$Classified_Celltype <- AML_scATA_meta$predictedGroup_Un

maincelltypes <- c("HSPC", "Myeloid", "Lymphoid", "Erythroid", "Other")
subcelltypes <- c(HSPC, Myeloid, Lymphoid, Erythroid_Basophil, Other)

res_ct_order_manu <- c(HSPC, Myeloid, Lymphoid, Erythroid_Basophil, Other)
res_ct_order_manu <- c("HSPC", "Myeloid", "Lymphoid", "Erythroid", "Other")

AML_scRNA_meta_DX <- AML_scRNA_meta %>%
  dplyr::filter(grepl("DX", Library_ID))

AML_scRNA_meta_REL <- AML_scRNA_meta %>%
  dplyr::filter(grepl("REL", Library_ID))

AML_scATA_meta_DX <- AML_scATA_meta %>%
  dplyr::filter(grepl("DX", Sample))

AML_scATA_meta_REL <- AML_scATA_meta %>%
  dplyr::filter(grepl("REL", Sample))

# common_sample_pairs, ct_forComparision, CSM_list_RNA_DX, CSM_list_RNA_REL, AML_meta_RNA
data_mutationCelltype_similarity_scRNA_DX_REL <- cal_Similarity_scRNA_DX_REL(common_sample_pairs = c("A
ct_forComparision = subcelltypes,
CSM_list_RNA_DX = AML_scRNA_CSM_DX_lis

```



```

message(sample)
csm_tmp <- CSM_list[[sample]]
for (i in ct_forComparision){
  ct_i <- i
  cell_barcode_meta_i <- meta_data %>%
    mutate(CellTag = rownames(.)) %>%
    dplyr::filter(Classified_Celltype == ct_i & Library_ID == sample) %>%
    pull(CellTag)
  cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
    sub("_L[0-9]+_", "_", .) %>%
    sub("_(.*)_ ", "_\\1.", .) %>%
    paste0(., "-1") %>%
    sub("\\.", "#", x = .)
  cell_barcode_csm_selected_i <- rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_r
  mut_site_i <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_i)$Mut
  if (length(cell_barcode_meta_i) <= 10){
    mut_site_i <- "Empty"
  }
  data_mutationCelltype_sites_list[[sample]][[ct_i]] <- mut_site_i
}
}
return(data_mutationCelltype_sites_list)
}

get_scATA_mutSites_byStages <- function(CSM_list, meta_data){
  common_sample_pairs <- unique(names(CSM_list))
  ct_forComparision <- unique(meta_data$Classified_Celltype)
  ct_forComparision <- setdiff(ct_forComparision, "Unknown")
  print(ct_forComparision)
  data_mutationCelltype_sites_list <- list()
  for (sample in common_sample_pairs){
    message(sample)
    csm_tmp <- CSM_list[[sample]]
    for (i in ct_forComparision){
      ct_i <- i
      cell_barcode_meta_i <- meta_data %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(Classified_Celltype == ct_i & Sample == sample) %>%
        pull(CellTag)
      cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)_ ", "_\\1.", .) %>%
        paste0(., "-1") %>%
        sub("\\.", "#", x = .)
      cell_barcode_csm_selected_i <- rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_r
      mut_site_i <- retrieveLocsByCells(csm_tmp, cell_barcode_csm_selected_i)$Mut
      if (length(cell_barcode_meta_i) <= 10){
        mut_site_i <- "Empty"
      }
      data_mutationCelltype_sites_list[[sample]][[ct_i]] <- mut_site_i
    }
  }
  return(data_mutationCelltype_sites_list)
}

```

```

}

# scRNA
# AML_CSM_DX_list
AML_scRNA_CSM_DX_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_DX_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_DX_list)

# AML_CSM_REM_list
AML_scRNA_CSM_REM_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REM_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_REM_list)

# AML_CSM_REL_list
AML_scRNA_CSM_REL_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REL_list <- Filter(function(x) nrow(x) > 0, AML_scRNA_CSM_REL_list)

# scATA
# AML_CSM_DX_list
AML_scATA_CSM_DX_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_DX_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_DX_list)

# AML_CSM_REM_list
AML_scATA_CSM_REM_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REM_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_REM_list)

# AML_CSM_REL_list
AML_scATA_CSM_REL_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REL_list <- Filter(function(x) nrow(x) > 0, AML_scATA_CSM_REL_list)

HSPC <- c("HSC", "Progenitor")
Myeloid <- c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid <- c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive", "CD4.Memory", "CD8.Naive", "CD8.Memory", "CD8")
Erythroid_Basophil <- c("Early.Erythrocyte", "Late.Erythrocyte")
Other <- c("Early.Basophil", "Plasma")

AML_scRNA_meta <- AML_scRNA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      Classified_Celltype %in% HSPC ~ "HSPC",
      Classified_Celltype %in% Myeloid ~ "Myeloid",
      Classified_Celltype %in% Lymphoid ~ "Lymphoid",
      Classified_Celltype %in% Erythroid_Basophil ~ "Erythroid",

```

```

      Classified_Celltype %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta <- AML_scATA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      predictedGroup_Un %in% HSPC ~ "HSPC",
      predictedGroup_Un %in% Myeloid ~ "Myeloid",
      predictedGroup_Un %in% Lymphoid ~ "Lymphoid",
      predictedGroup_Un %in% Erythroid_Basophil ~ "Erythroid",
      predictedGroup_Un %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta$Classified_Celltype <- AML_scATA_meta$predictedGroup_Un

maincelltypes <- c("HSPC", "Myeloid", "Lymphoid", "Erythroid", "Other")
subcelltypes <- c(HSPC, Myeloid, Lymphoid, Erythroid_Basophil, Other)

AML_scrNA_meta_DX <- AML_scrNA_meta %>%
  dplyr::filter(grepl("DX", Library_ID))

AML_scrNA_meta_REL <- AML_scrNA_meta %>%
  dplyr::filter(grepl("REL", Library_ID))

AML_scATA_meta_DX <- AML_scATA_meta %>%
  dplyr::filter(grepl("DX", Sample))

AML_scATA_meta_REL <- AML_scATA_meta %>%
  dplyr::filter(grepl("REL", Sample))

AML_scrNA_meta_DX$Classified_CelltypeQ <- paste(AML_scrNA_meta_DX$Classified_Celltype, "DX", sep = "_")
AML_scrNA_meta_REL$Classified_CelltypeQ <- paste(AML_scrNA_meta_REL$Classified_Celltype, "REL", sep = "_")
AML_scrNA_meta_DX$maincelltypeQ <- paste(AML_scrNA_meta_DX$maincelltype, "DX", sep = "_")
AML_scrNA_meta_REL$maincelltypeQ <- paste(AML_scrNA_meta_REL$maincelltype, "REL", sep = "_")

AML_scATA_meta_DX$Classified_CelltypeQ <- paste(AML_scATA_meta_DX$Classified_Celltype, "DX", sep = "_")
AML_scATA_meta_REL$Classified_CelltypeQ <- paste(AML_scATA_meta_REL$Classified_Celltype, "REL", sep = "_")
AML_scATA_meta_DX$maincelltypeQ <- paste(AML_scATA_meta_DX$maincelltype, "DX", sep = "_")
AML_scATA_meta_REL$maincelltypeQ <- paste(AML_scATA_meta_REL$maincelltype, "REL", sep = "_")

AML_scrNA_meta_DX_REL <- rbind(AML_scrNA_meta_DX, AML_scrNA_meta_REL)
AML_scATA_meta_DX_REL <- rbind(AML_scATA_meta_DX, AML_scATA_meta_REL)

selected_samples_id <- c("AML4", "AML11", "AML16", "AML17", "AML18", "AML24")
# scrNA
AML_scrNA_CSM_DX_select_list <- AML_scrNA_CSM_DX_list[names(AML_scrNA_CSM_DX_list) %in% selected_samples_id]
names(AML_scrNA_CSM_DX_select_list) <- paste(selected_samples_id, "DX", sep = "_")
AML_scrNA_CSM_REL_select_list <- AML_scrNA_CSM_REL_list[names(AML_scrNA_CSM_REL_list) %in% selected_samples_id]
names(AML_scrNA_CSM_REL_select_list) <- paste(selected_samples_id, "REL", sep = "_")

```

```

AML_scRNA_CSM_DX_REL_select_list <- c(AML_scRNA_CSM_DX_select_list, AML_scRNA_CSM_REL_select_list)

# scATA
AML_scATA_CSM_DX_select_list <- AML_scATA_CSM_DX_list[names(AML_scATA_CSM_DX_list) %in% selected_samples_id]
names(AML_scATA_CSM_DX_select_list) <- paste(selected_samples_id, "DX", sep = "_")
AML_scATA_CSM_REL_select_list <- AML_scATA_CSM_REL_list[names(AML_scATA_CSM_REL_list) %in% selected_samples_id]
names(AML_scATA_CSM_REL_select_list) <- paste(selected_samples_id, "REL", sep = "_")
AML_scATA_CSM_DX_REL_select_list <- c(AML_scATA_CSM_DX_select_list, AML_scATA_CSM_REL_select_list)

data_mutationCelltype_mutSites_scRNA_byStage <- get_scRNA_mutSites_byStages(AML_scRNA_CSM_DX_REL_select_list)
data_mutationCelltype_mutSites_scATA_byStage <- get_scATA_mutSites_byStages(AML_scATA_CSM_DX_REL_select_list)

data_mutationCelltype_similarity_scRNA_byStage_list <- list()
data_mutationCelltype_similarity_scATA_byStage_list <- list()
common_sample_pairs <- selected_samples_id
common_sample_stages <- c("DX", "REL")
ct_forComparision <- subcelltypes

for (sample in common_sample_pairs){
  message(sample)
  row_col_data_Names <- c(paste(ct_forComparision, "DX", sep = "_"), paste(ct_forComparision, "REL", sep = "_"))
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(row_col_data_Names),
                                                    ncol = length(row_col_data_Names)))

  rownames(mut_cellTypeAnn_similarity) <- row_col_data_Names
  colnames(mut_cellTypeAnn_similarity) <- row_col_data_Names
  for (i in 1:nrow(mut_cellTypeAnn_similarity)){
    ct_stage_i <- rownames(mut_cellTypeAnn_similarity)[i]
    ct_i <- strsplit(ct_stage_i, "_")[[1]][1]
    stage_i <- strsplit(ct_stage_i, "_")[[1]][2]
    mut_site_i <- data_mutationCelltype_mutSites_scRNA_byStage[[paste(sample, stage_i, sep = "_")]][[ct_i]]
    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      ct_stage_j <- colnames(mut_cellTypeAnn_similarity)[j]
      ct_j <- strsplit(ct_stage_j, "_")[[1]][1]
      stage_j <- strsplit(ct_stage_j, "_")[[1]][2]
      mut_site_j <- data_mutationCelltype_mutSites_scRNA_byStage[[paste(sample, stage_j, sep = "_")]][[ct_j]]
      similarity_score <- ifelse(
        length(unique(c(mut_site_i, mut_site_j))) == 0,
        0,
        length(unique(intersect(mut_site_i, mut_site_j))) / length(unique(c(mut_site_i, mut_site_j)))
      )
      if (length(mut_site_i) == 1 | length(mut_site_j) == 1){
        similarity_score <- 0
      }

      mut_cellTypeAnn_similarity[i, j] <- similarity_score
    }
  }
  data_mutationCelltype_similarity_scRNA_byStage_list[[sample]] <- mut_cellTypeAnn_similarity
}

for (sample in common_sample_pairs){
  message(sample)
  row_col_data_Names <- c(paste(ct_forComparision, "DX", sep = "_"), paste(ct_forComparision, "REL", sep = "_"))

```

```

mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow = length(row_col_data_Names),
                                                    ncol = length(row_col_data_Names)))
rownames(mut_cellTypeAnn_similarity) <- row_col_data_Names
colnames(mut_cellTypeAnn_similarity) <- row_col_data_Names
for (i in 1:nrow(mut_cellTypeAnn_similarity)){
  ct_stage_i <- rownames(mut_cellTypeAnn_similarity)[i]
  ct_i <- strsplit(ct_stage_i, "_")[[1]][1]
  stage_i <- strsplit(ct_stage_i, "_")[[1]][2]
  mut_site_i <- data_mutationCelltype_mutSites_scATA_byStage[[paste(sample, stage_i, sep = "_")]][[ct_i]]
  for (j in 1:ncol(mut_cellTypeAnn_similarity)){
    ct_stage_j <- colnames(mut_cellTypeAnn_similarity)[j]
    ct_j <- strsplit(ct_stage_j, "_")[[1]][1]
    stage_j <- strsplit(ct_stage_j, "_")[[1]][2]
    mut_site_j <- data_mutationCelltype_mutSites_scATA_byStage[[paste(sample, stage_j, sep = "_")]][[ct_j]]
    similarity_score <- ifelse(
      length(unique(c(mut_site_i, mut_site_j))) == 0,
      0,
      length(unique(intersect(mut_site_i, mut_site_j))) / length(unique(c(mut_site_i, mut_site_j)))
    )
    if (length(mut_site_i) == 1 | length(mut_site_j) == 1){
      similarity_score <- 0
    }
    mut_cellTypeAnn_similarity[i, j] <- similarity_score
  }
}
data_mutationCelltype_similarity_scATA_byStage_list[[sample]] <- mut_cellTypeAnn_similarity
}

```